wk02 — SE·PL 정답 및 해설

- 출제자: 승혜
- 문제지: `wk02_SE, PL_승혜.md`
- 출제 과목: SE(소프트웨어공학), PL(프로그래밍언어)
- 총 2문항 / 20점
- 채점 방식: 핵심 키워드 및 논리적 설명에 따른 부분점수 인정

문제 1. SE — 소프트웨어공학 (10점)

(1) 응집도와 결합도 [3점]

모범답안

소프트웨어 설계에서는 응집도(Cohesion)를 높이고 결합도(Coupling)를 낮추는 것이 바람직하다. 높은 응집도는 하나의 모듈이 밀접하게 관련된 기능을 수행하도록 하며, 낮은 결합도는 모듈 간 의존성을 줄인다. 이를 통해 기능 변경 시 다른 모듈에 미치는 영향을 최소화하여 유지보수성을 높일 수 있다.

핵심 키워드 - 높은 응집도 - 낮은 결합도 - 변경 영향 범위 최소화

채점 기준 - 응집도를 높여야 한다고 기술: 1점 - 결합도를 낮춰야 한다고 기술: 1점 - 모듈 독립성, 변경 영향 감소 또는 유지보수성 향상 설명: 1점

(2) SOLID 설계 원칙 [3점]

모범답안

의존관계 역전 원칙(DIP, Dependency Inversion Principle)이다. 상위 수준 모듈은 구체적인 하위 구현이 아닌 추상화에 의존해야 한다는 원칙이다. 이를 적용하면 알림 발송 방식 등 구체적인 구현이 변경되더라도 이를 사용하는 상위 모듈의 수정 필요성을 줄일 수 있어 유지보수성이 향상된다.

핵심 키워드 - 의존관계 역전 원칙(DIP) - 추상화에 의존 - 구현 변경 영향 최소화

채점 기준 - DIP 또는 의존관계 역전 원칙 명시: 1점 - 구체적 구현이 아닌 추상화에 의존하는 구조 설명: 1점 - 구현 변경 시 수정 범위 감소 및 유지보수성 향상 설명: 1점

보충 해설

인터페이스를 사용한다는 사실만으로 DIP가 자동으로 충족되는 것은 아니다. 상위 정책 모듈과 하위 구현 모듈이 공통 추상화에 의존하도록 설계하는 것이 핵심이다.

(3) 소프트웨어 테스트 유형 [4점]

모범답안

① 통합 테스트(Integration Test): 신용평가, 승인 처리, 알림 발송 등 개별 모듈 간 상호작용과 데이터 전달이 정상적으로 수행되는지 검증한다.

② 회귀 테스트(Regression Test): 승인 기준 등 기존 기능을 수정한 이후 다른 기능에 새로운 결함이 발생하지 않았는지 검증한다.

핵심 키워드 - 통합 테스트: 모듈 간 인터페이스 및 상호작용 검증 - 회귀 테스트: 변경 이후 기존 기능의 정상 동작 검증

채점 기준 - 통합 테스트 명칭: 1점 - 통합 테스트 목적 설명: 1점 - 회귀 테스트 명칭: 1점 - 회귀 테스트 목적 설명: 1점

부분점수 유의사항

단위 테스트(Unit Test) 등 다른 테스트 유형을 제시하더라도 해당 상황에서의 검증 대상과 목적을 타당하게 설명했다면 인정할 수 있다. 동일한 검증 목적을 중복 기술한 경우에는 중복 점수를 부여하지 않는다.

문제 2. PL — 프로그래밍언어 (10점)

(1) 프로그램 출력 결과 [3점]

정답

```
1200
200
700
```

채점 기준 - 첫 번째 출력값 1200: 1점 - 두 번째 출력값 200: 1점 - 세 번째 출력값 700: 1점

(2) Java 매개변수 전달 방식 [3점]

모범답안

Java는 모든 매개변수를 값에 의한 전달(Call by Value) 방식으로 전달하며, 객체를 전달할 때는 참조값이 복사된다. 따라서 `a = new Account(500)`은 메서드 내부의 지역 참조변수 a가 새로운 객체를 가리키게 할 뿐, 호출자의 x가 참조하는 객체를 변경하지 않는다. 이후 `a.balance`를 수정해도 `x.balance`에는 영향을 주지 않는다.

핵심 키워드 - Call by Value - 객체 참조값의 복사 - 지역 참조변수 재할당 - 기존 객체와 새로운 객체 구분

채점 기준 - 값에 의한 전달 명시: 1점 - 객체 참조값이 복사됨을 설명: 1점 - 지역변수 재할당이 호출자의 참조 대상을 바꾸지 않음을 설명: 1점

보충 해설

`a.balance += 100`은 기존 객체의 필드를 수정하므로 `x.balance`도 200이 된다. 반면 `a = new Account(500)` 이후의 필드 변경은 새 객체에만 적용된다.

(3) finally 실행과 반환값 및 객체 상태 [4점]

모범답안

`return a.balance + b.balance`에서 반환식은 `finally` 실행 전에 평가되므로 반환값은 500 + 700 = 1200으로 결정된다. 이후 `finally`의 `a.balance += 200`은 새로 생성된 객체를 변경하므로 반환값과 `x.balance`에는 영향을 주지 않는다.

또한 `b = new Account(1000)`은 지역 참조변수 b의 참조 대상을 변경할 뿐, 호출자 변수 y가 참조하는 기존 객체를 바꾸지 않는다. 따라서 `y.balance`는 700을 유지한다.

핵심 키워드 - return 표현식 선평가 - finally 실행 - 지역 참조변수 재할당 - 원본 객체 상태 유지

채점 기준 - return 표현식이 finally보다 먼저 평가됨을 설명: 1점 - 반환값 1200이 유지되는 이유 설명: 1점 - b의 재할당이 지역 참조변수에만 적용됨을 설명: 1점 - y.balance가 700으로 유지되는 이유 설명: 1점

전체 실행 흐름

단계	실행 코드	결과
1	`a.balance += 100`	x.balance = 200
2	`a = new Account(500)`	a가 새 객체 참조
3	`b.balance += a.balance`	y.balance = 700
4	`return a.balance + b.balance`	반환값 1200 평가
5	`a.balance += 200`	새 객체의 balance = 700
6	`b = new Account(1000)`	지역 참조변수 b만 변경
7	메서드 종료	result=1200, x.balance=200, y.balance=700

보충 해설

일반적인 정상 종료 상황에서 finally는 return 이후 메서드가 실제 종료되기 전에 실행된다. 이 코드에서는 finally에 별도의 return이 없으므로 앞서 평가된 반환값 1200이 유지된다.

전체 채점표

문제	소문항	배점
SE	(1) 응집도·결합도	3점
SE	(2) DIP 설계 원칙	3점
SE	(3) 테스트 유형	4점
PL	(1) 출력 결과	3점
PL	(2) 매개변수 전달 방식	3점
PL	(3) finally 및 참조 분석	4점
합계	**2문항, 소문항 6개**	**20점**

공통 채점 원칙

- 핵심 개념과 인과관계가 정확하면 모범답안과 표현이 달라도 정답으로 인정한다.
- 단순 키워드 나열과 원리 설명을 구분하여 부분점수를 부여한다.
- 기술적으로 잘못된 설명은 해당 평가 요소에서 감점한다.
- 동일한 의미의 표현은 중복 채점하지 않는다.